



UNIVERSIDAD NACIONAL EXPERIMENTAL DEL TÁCHIRA
VICERRECTORADO ACADÉMICO
COMUNICACIONES II

Parcial III: Diseño, Adversidad y Robustecimiento de Protocolos de Comunicación

Jessica Ramírez CI: 23513289

Enderson Chávez CI: 21542645

Mauro León CI: 27239906

Agosto de 2026

Informe Técnico

A. Especificación de Protocolo

Para lograr la comunicación entre cliente y servidor se diseñó un protocolo de capa de aplicación, lo presentamos en dos partes, una versión inicial del proyecto antes de los ataques y el robustecimiento y una versión robusta en la que se describen las mejoras.

Versión Inicial del proyecto

Capa de transporte y comunicación:

Se usó **TCP** como protocolo base para poder garantizar una conexión orientada a flujo, ya que esta es justo una de las características de TCP. Sobre el direccionamiento, este se realizó en un loopback, es decir localmente sobre la **Dirección IPv4 127.0.0.1** a través del **Puerto 5000**.

Codificación y Serialización:

Se usó el formato de datos **JSON** para serializar los mensajes en texto plano, y se usó la **codificación UTF-8**, para no tener problemas en la interpretación de los caracteres en flujo de bytes.

Tramas:

Como usamos TCP que transmite los datos en un flujo continuo, se implementó un mecanismo de delimitación explícito. Además, se utilizó el carácter **\n** al final de cada **JSON** serializado. Los datos que entran son almacenados por el servidor en un búfer temporal y extrae las tramas individuales únicamente cuando se encuentra con el delimitador **\n**.

Estructura del Mensaje:

En cada trama transmitida es un objeto **JSON** que contiene tres campos obligatorios:

- Tipo: Que es un *string* este actúa como identificador del mensaje para saber qué acción se debe ejecutar.
- Longitud: Que es un *entero* e indica el tamaño o número de caracteres que contiene el cuerpo del mensaje
- Datos: Que es un *string* que representa el **payload** del mensaje transmitido.

Flujo de Comunicación:

1. Inicia **conexión** TCP de cliente a servidor
2. El cliente envía una **petición**, una trama inicial de tipo "SALUDO", incluyendo un texto de prueba en el *payload*.
3. Procesamiento y respuesta, el servidor extrae la trama de búfer, decodifica el JSON, muestra los datos en la consola y responde con una trama de tipo "RESPUESTA_OK", que indica que todo salió bien.
4. Cierre, después de recibir y leer la respuesta del servidor el cliente destruye el socket y cierra la conexión.

Versión Robusta:

Luego de estudiar el impacto que genero cada uno de los ataques, se tomaron medidas y se reestructuraron los protocolos para hacer frente a posibles fallos generados por ataques:

Estructura del mensaje:

Se añadió a la estructura anterior de la trama JSON los siguientes campos:

- Hash: que es un *string* que contiene una huella digital generado por el algoritmo criptográfico SHA-256, a partir de texto que este en *datos*.

Para la tolerancia a fallas:

- Se aplica hash SHA-256 al *payload*, si no coinciden las huellas generadas por el cliente y el servidor se responde un NACK indicando un archivo corrupto.
- Se configuro un Timeout de 2000ms como tiempo límite para asumir una falla en el canal
- Uso del esquema Stop-and-wait en el contexto de la retransmisión, el cliente reenvía automáticamente el mensaje si detecta algún error de red, timeout o recibe un NACK
- Se establece un limite de reintentos de 3 para evitar bucles infinitos se ha caído la red

B. Bitácora de Adversidad

Herramientas Utilizadas

Clumsy 0.3: Para la inyección de fallas. Se configuro el puerto 5000. Aplicando el filtro:

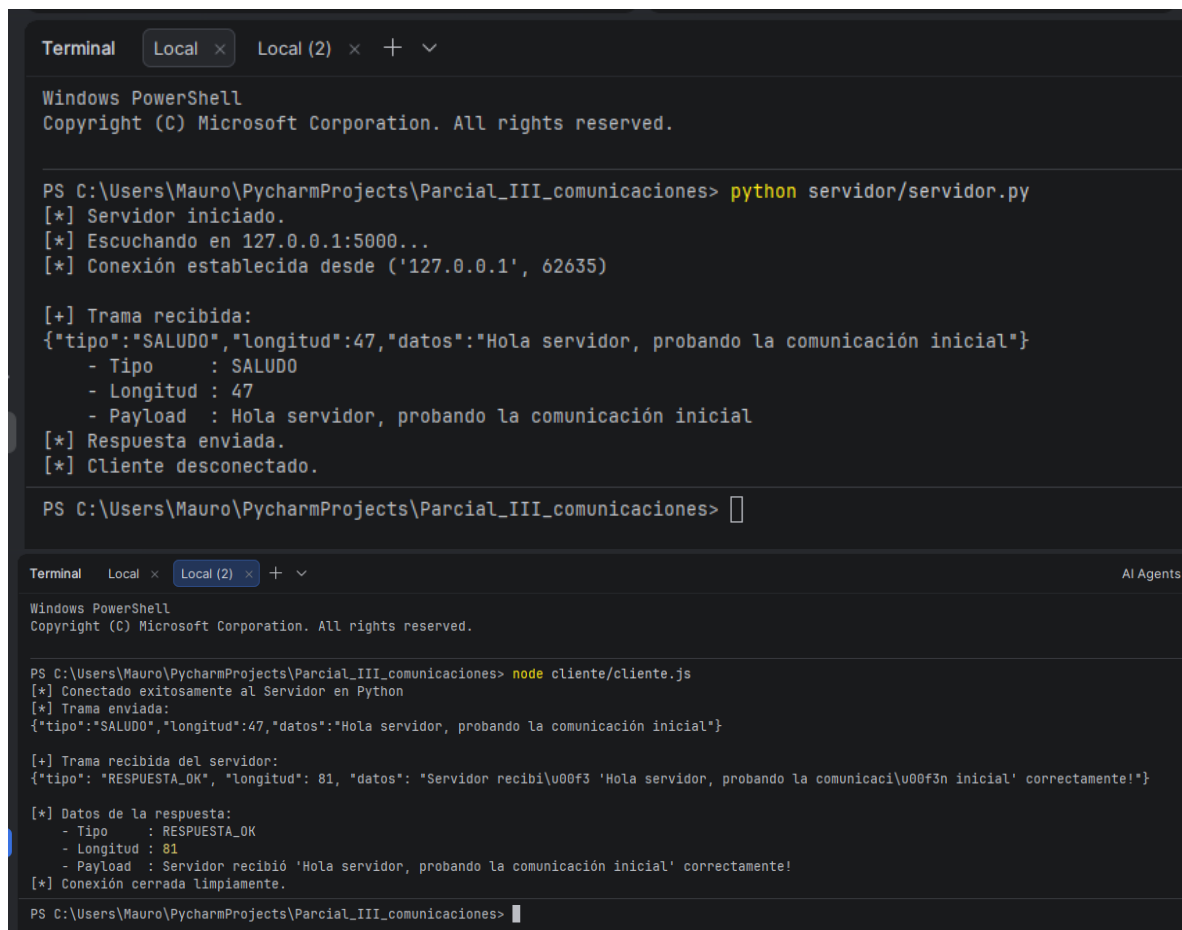
outbound and (tcp.DstPort == 5000 or tcp.SrcPort == 5000)

Wireshark: Para el monitoreo y captura

Consola: registrar respuestas y errores en la comunicación.

Prueba inicial

La prueba inicial se realizó sin interferencias, ejecutando el servidor en Python y el cliente en Node.js mediante 127.0.0.1:5000. El cliente se conectó correctamente, envió la trama SALUDO y el servidor recibió y procesó sus campos sin errores. Finalmente, el servidor respondió con RESPUESTA_OK, confirmando que la comunicación básica entre cliente y servidor funcionaba correctamente en un entorno controlado.



```
Terminal Local x Local (2) x + v

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> python servidor/servidor.py
[*] Servidor iniciado.
[*] Escuchando en 127.0.0.1:5000...
[*] Conexión establecida desde ('127.0.0.1', 62635)

[+] Trama recibida:
{"tipo":"SALUDO","longitud":47,"datos":"Hola servidor, probando la comunicación inicial"}
  - Tipo      : SALUDO
  - Longitud  : 47
  - Payload   : Hola servidor, probando la comunicación inicial
[*] Respuesta enviada.
[*] Cliente desconectado.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones>

Terminal Local x Local (2) x + v AI Agents

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> node cliente/cliente.js
[*] Conectado exitosamente al Servidor en Python
[*] Trama enviada:
{"tipo":"SALUDO","longitud":47,"datos":"Hola servidor, probando la comunicación inicial"}

[+] Trama recibida del servidor:
{"tipo": "RESPUESTA_OK", "longitud": 81, "datos": "Servidor recibí\u00f3 'Hola servidor, probando la comunicaci\u00f3n inicial' correctamente!"}

[*] Datos de la respuesta:
  - Tipo      : RESPUESTA_OK
  - Longitud  : 81
  - Payload   : Servidor recibió 'Hola servidor, probando la comunicación inicial' correctamente!
[*] Conexión cerrada limpiamente.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones>
```

Inyección de Fallas y Análisis de Debilidades

Para evaluar el comportamiento del protocolo ante condiciones adversas se utilizó Clumsy, configurado para trabajar sobre el tráfico de localhost. Se utilizó el filtro: Outbound and loopback y el preset: localhost IPv4. Las pruebas se realizaron de manera individual, activando una condición de falla a la vez para poder identificar claramente sus efectos. Se evaluaron principalmente latencia, pérdida de paquetes y corrupción de datos, cumpliendo con el requisito de aplicar al menos tres condiciones de falla.

1.Prueba de latencia

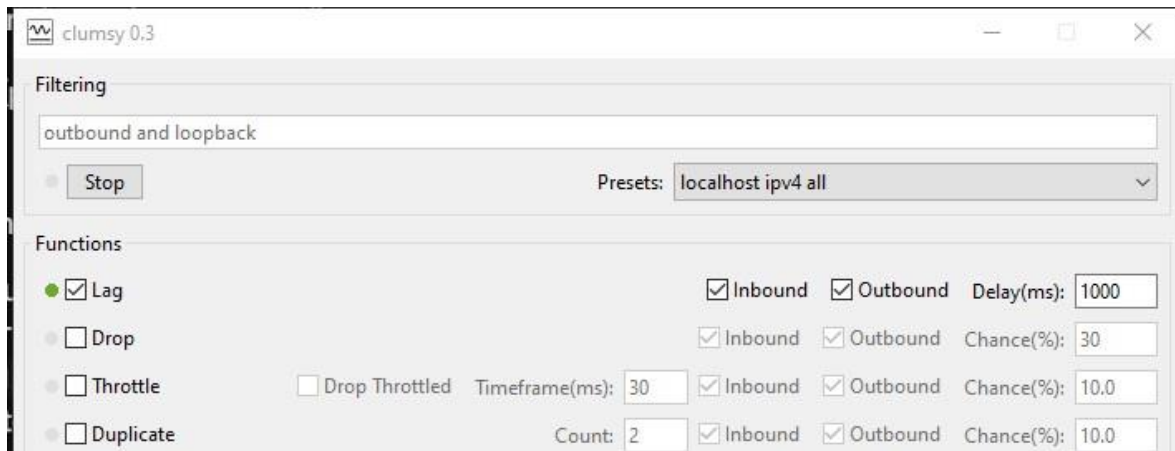
Para simular una condición de latencia se utilizó la opción Lag de Clumsy. inicialmente se configuró un retraso de aproximadamente 100 ms.

Con esta configuración, el cliente consiguió establecer la conexión y enviar la trama correctamente. El servidor recibió el mensaje completo y generó la respuesta correspondiente. El resultado fue:

```
PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> python servidor/servidor.py
[*] Servidor iniciado.
[*] Escuchando en 127.0.0.1:5000...
[*] Conexión establecida desde ('127.0.0.1', 62635)

[+] Trama recibida:
{"tipo":"SALUDO","longitud":47,"datos":"Hola servidor, probando la comunicación inicial"}
  - Tipo      : SALUDO
  - Longitud  : 47
  - Payload   : Hola servidor, probando la comunicación inicial
[*] Respuesta enviada.
[*] Cliente desconectado.
```

La comunicación continuó funcionando correctamente, aunque con un retraso adicional. Posteriormente se realizó una prueba utilizando un valor de latencia considerablemente mayor de 1000 ms.



En este caso el cliente presentó:

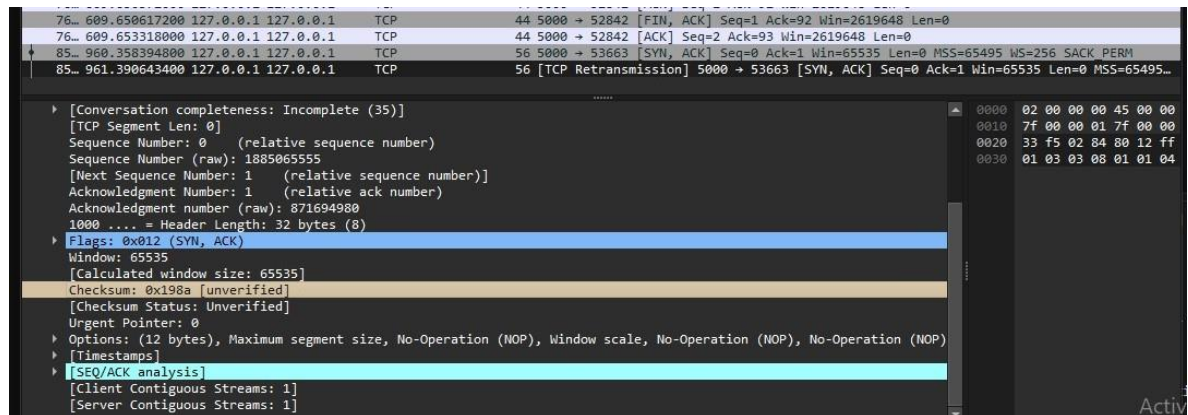
[-] Error en el cliente: connect ETIMEDOUT 127.0.0.1:5000

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> node cliente/cliente.js
[-] Error en el cliente: connect ETIMEDOUT 127.0.0.1:5000
[*] Conexión cerrada limpiamente.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> 
```

Esto demuestra que, cuando el retraso supera el tiempo que la aplicación o el sistema está dispuesto a esperar, la conexión puede terminar en un timeout. Observando además la captura hecha por wireshark queda evidencia que el TCP se rompió, el servidor envió su respuesta SYN, ACK pero el ACK final del cliente tanto que el servidor asumió que se había perdido, disparando así la retransmisión TCP que se observa en la captura.

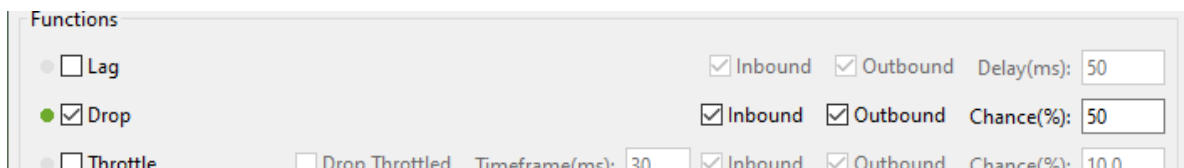


Debilidad encontrada: el protocolo de aplicación inicial no posee mecanismos propios para controlar tiempos de espera ni para realizar una retransmisión controlada cuando una operación no recibe respuesta.

1) Prueba de pérdida de paquetes

La pérdida de paquetes fue simulada mediante la opción Drop de Clumsy. Primero se utilizó una probabilidad de pérdida del 10 %. En esta prueba la comunicación terminó correctamente. El servidor recibió la trama y el cliente obtuvo la respuesta.

Esto permitió observar que TCP posee mecanismos propios para recuperarse de determinadas pérdidas, por lo que una pérdida moderada no necesariamente produce un error visible en la aplicación. Posteriormente se aumentó la condición adversa a 50 % de pérdida.



En esta prueba el servidor permaneció esperando una conexión:

- [*] Servidor iniciado.
- [*] Escuchando en 127.0.0.1:5000...



mientras que el cliente terminó mostrando:

[~] Error en el cliente: connect ETIMEDOUT 127.0.0.1:5000



```
Terminal Local (2) x Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> node cliente/cliente.js
[~] Error en el cliente: connect ETIMEDOUT 127.0.0.1:5000
[*] Conexión cerrada limpiamente.

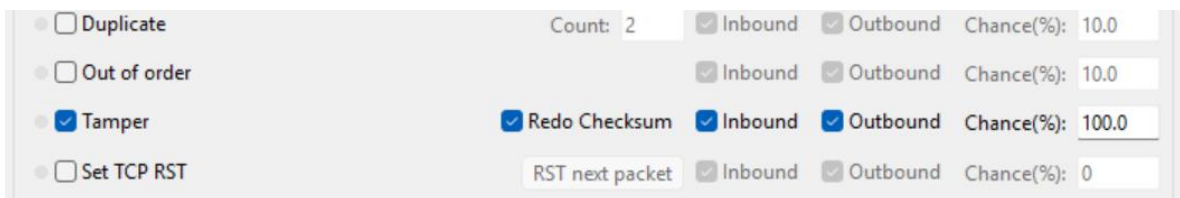
PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> |
```

En este caso la pérdida fue suficiente para impedir que el establecimiento de la conexión TCP se completara dentro del tiempo de espera.

Debilidad encontrada: el protocolo de aplicación no posee un mecanismo propio de confirmación, timeout y retransmisión. Aunque TCP proporciona recuperación a nivel de transporte, la aplicación no tiene control sobre este proceso ni un mecanismo propio para determinar que una trama específica debe ser retransmitida.

2) Prueba de corrupción de datos

Para comprobar el comportamiento ante datos alterados se utilizó la función Tamper de Clumsy con una probabilidad del 100 %.



En esta prueba el cliente consiguió establecer la conexión y enviar la trama:

```
{"tipo":"SALUDO","longitud":47,"datos":"Hola servidor, probando la comunicación inicial"}
```

Sin embargo, al modificar los datos transmitidos, el servidor recibió bytes que no podían ser interpretados correctamente como UTF-8. El servidor terminó con la excepción:

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 41

```
Terminal Local (2) x Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

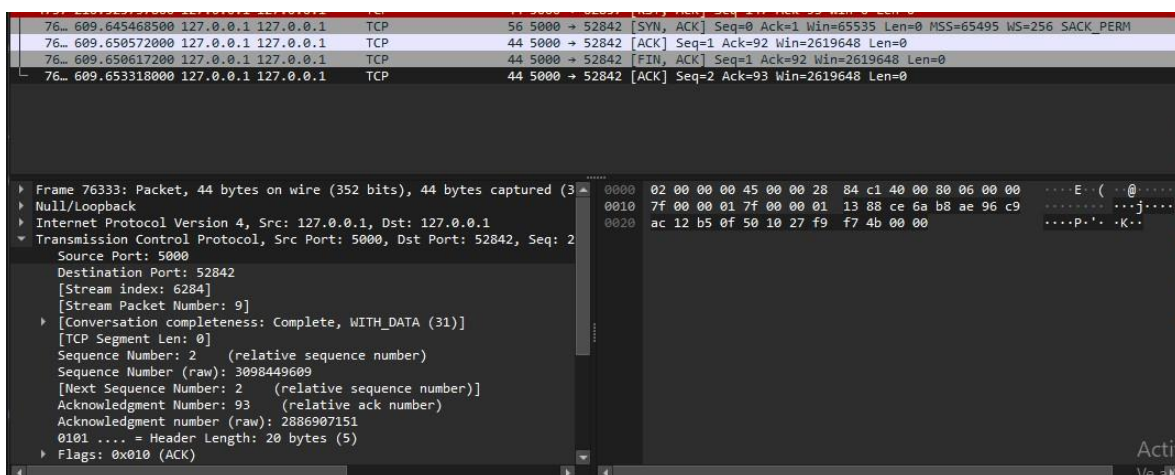
PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones> python servidor/servidor.py
[*] Servidor iniciado.
[*] Escuchando en 127.0.0.1:5000...
[*] Conexión establecida desde ('127.0.0.1', 59148)
Traceback (most recent call last):
  File "C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones\servidor\servidor.py", line 94, in <module>
    iniciar_servidor()
    ~~~~~^~~~~~
  File "C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones\servidor\servidor.py", line 39, in iniciar_servidor
    buffer += data.decode('utf-8')
    ~~~~~^~
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe7 in position 41: invalid continuation byte

PS C:\Users\Mauro\PycharmProjects\Parcial_III_comunicaciones>
```

El problema se produjo en el procesamiento:

```
buffer += data.decode('utf-8')
```

Esto significa que el servidor intentó convertir directamente los bytes recibidos a UTF-8 sin comprobar previamente que los datos fueran válidos. Como consecuencia, la aplicación servidor terminó inesperadamente ante una alteración del contenido recibido. Esto se confirma en WireShark donde se observa que el ataque alteró los bits del mensaje, pero recalculó la suma de comprobación de la cabecera TCP. Por lo tanto, Wireshark y el sistema operativo no detectan ningún error a nivel de transporte. TCP asume que el paquete es válido y se lo entrega directamente a la aplicación.



```
76_ 609.645468500 127.0.0.1 127.0.0.1 TCP 56 5000 -> 52842 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
76_ 609.650572000 127.0.0.1 127.0.0.1 TCP 44 5000 -> 52842 [ACK] Seq=1 Ack=92 Win=2619648 Len=0
76_ 609.650617200 127.0.0.1 127.0.0.1 TCP 44 5000 -> 52842 [FIN, ACK] Seq=1 Ack=92 Win=2619648 Len=0
76_ 609.653318000 127.0.0.1 127.0.0.1 TCP 44 5000 -> 52842 [ACK] Seq=2 Ack=93 Win=2619648 Len=0

Frame 76333: Packet, 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 5000, Dst Port: 52842, Seq: 2
  Source Port: 5000
  Destination Port: 52842
  [Stream index: 6284]
  [Stream Packet Number: 9]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 0]
  Sequence Number: 2 (relative sequence number)
  Sequence Number (raw): 3098449609
  [Next Sequence Number: 2 (relative sequence number)]
  Acknowledgment Number: 93 (relative ack number)
  Acknowledgment number (raw): 2886907151
  0101 .... = Header Length: 20 bytes (5)
  Flags: 0x010 (ACK)
  0000 02 00 00 00 45 00 00 28 84 c1 40 00 00 06 00 00 ...E.(.0....
  0010 7f 00 00 01 7f 00 00 01 13 88 ce 6a b8 ae 96 c9 .....j....
  0020 ac 12 b5 0f 50 10 27 f9 f7 4b 00 00 .....p.'..k..
```

Debilidad encontrada: el protocolo inicial no posee mecanismos de verificación de integridad y tampoco tiene un manejo adecuado de excepciones para datos corruptos. Una alteración del canal puede provocar que la aplicación se cierre en lugar de descartar la trama y continuar funcionando.

Matriz de Errores y Soluciones:

Falla Inyectada	Comportamiento Inicial	Solución Implementada
Latencia / Retardo (Lag 1000ms)	El cliente se quedaba en un estado de espera infinito esperando la respuesta del servidor. Esto rompía la sincronización de TCP causando retransmisiones innecesarias.	Se integró un Timeout de 2000 ms (2 segundos) en el cliente. Si no se recibe respuesta en este lapso, el sistema interrumpe la espera y gestiona el fallo.
Pérdida de Paquetes (Drop 50%)	El descarte de tramas provocaba que el servidor nunca recibiera la orden o que el cliente nunca recibiera la respuesta, desencadenando desconexiones inesperadas (paquetes RST) o cuelgues.	Se aplicó un modelo Stop-and-Wait . Ante la caída de red o al agotarse el tiempo del temporizador, el cliente retransmite automáticamente la trama hasta un máximo de 3 intentos MAX_REINTENTOS .
Corrupción de Datos (Tamper 100%)	Como la alteración se hacía recalculando el Checksum de transporte, TCP entregaba el paquete dañado. El servidor colapsaba al intentar procesar un JSON malformado con caracteres ilegibles.	Se añadió un campo hash, algoritmo SHA-256 a la trama JSON. El servidor ahora valida el payload antes de procesarlo; si detecta alteraciones, rechaza el paquete respondiendo con un "NACK".